EECS581: Project 02 Documentation

Group 6 - Team Members: Kaitlyn Bedgood, Marie Biernacki, Ximena Bustos, Sabelli Antebi Delmas, Greeshma Kunduri, Lauren Lee, Jaydine Stiles

Program Purpose: The purpose of this project is to enhance Group 26's inherited Minesweeper system. The updated application features a custom UI enhancement (a session timer with high score tracking) and introduces an AI solver. The AI system is designed to alternate with the player or play the game by itself.

System Architecture Overview

Inherited System Components

File	game_state.py
Purpose	Manages the global game state and tracks victory/loss conditions.

Functions	
check_victory(grid)	Iterates through board to verify if all safe cells are revealed - Returns True and updates status to Victory if successful
set_game_over()	Sets the global status to LOSS
reset_game_status()	Resets the state back to PLAYING for a new session
get_game_status()	Returns the current game status string

File	main.py
Purpose	Connects the frontend GUI with the backend logic, handles initialization, and routes click events.

Functions	
get_mine_count()	Spawns a temporary Tkinter dialog asking the player how many mines they want. (No longer used, legacy code from original Project 1)
main()	Configures the backend, establishes event

	handler callbacks, launches the UI
refresh_status_bar()	Calculates and pushes current mine/flag counts to the interface
handle_left_click(row, col)	Passes left-click coordinates to the backend, triggers a visual re-render, and checks for game end
handle_right_click(row, col)	Passes right-click coordinates to the backend for flagging and triggers a re-render
handle_ai_turn()	Executes the AI move automatically, updates the UI, and creates an infinite loop of turns if Auto mode is selected.
report_status(status)	Prints the victory or loss state to the console

File	**interface.py**
Purpose	Handles the GUI using Tkinter, including window creation, cross-platform inputs, and grid rendering

Functions	
__init__	Initializes Tkinter window, loads visual assets, builds the UI layout including the status bar and game board
update_mine_count(count)	Updates the UI string showing remaining unflagged mines
update_score(flags_placed, total_mines)	Updates the UI string showing flags placed versus total mines
_load_scaled_image(...)	Loads and scales .png assets to fit tile constraints
_build_board(...)	Constructs the 2D grid of Tkinter buttons and binds cross-platform left/right-click events
set_title(...)	Configures the visual appearance (raised/sunken, image, text) of a single tile
render(grid)	Synchronizes the Tkinter grid visually with the underlying backend cell states
run()	Starts Tkinter main event loop

File	minesweeper.py
Purpose	Core backend logic managing the 10x10 grid, mine placement, flood-fill reveals, and cell objects.

Functions	
Cell (class)	Represents a single grid tile, tracking attributes like is_mine, is_revealed, is_flagged, and adjacent_mines.
configure(num_mines)	Sets the global initial mine count
onLeftClick(x,y)	Handles board initialization on the first click, reveals safe cells, and evaluates loss conditions or triggers recursive reveals
onRightClick(x,y)	Toggles the flagged status of a hidden cell and updates the remaining mine count
init_grid(firstclick_x, firstclick_y)	Builds the 2D array and protects the first click coordinate from housing a mine
place_mines(...)	Distribute mines randomly across the board
calculate_adjacent_mines()	Scans 3x3 radius around every safe cell to calculate neighbor mine counts.
reveal_adjacent_cells(x,y)	A recursive flood-fill algorithm that automatically opens safe neighbors of a 0 cell
game_over()	Triggers a loss state and forces all remaining mines to be revealed
reset()	Clears the grid and resets global variables for a new game
execute_ai_move()	Requests the next move coords from the selected AI solver and executes the click action

New AI & Configuration Files

File	setup_menu.py
Purpose	Provides a pre-game Tkinter configuration menu to capture user preferences for the game.

Functions	
get_game_config()	Launches a blocking UI window and returns a dictionary containing the chosen mine count, game mode, and AI difficulty
on_mode_change()	Disables the AI difficulty buttons if Solo mode is selected
on_start()	Callback function for the Start button that saves the user's selection and destroys the configuration window.

File	easy_solver.py
Purpose	Houses the base-level AI logic.

Functions	
get_next_move(grid)	Selects and returns a completely random valid, unrevealed, and unflagged cell.

File	medium_solver.py
Purpose	Houses the intermediate logical deduction AI.

Functions	
get_next_move(grid)	Analyzes adjacent cell numbers to explicitly flag guaranteed mines or reveal guaranteed safe cells, falling back to easy_solver.py if no logical moves exist.

File	hard_solver.py
Purpose	Houses the advanced pattern-recognition AI.

Functions	
get_next_move(grid)	Scans the board for 1-2-1 configurations before falling back to medium_solver.py rules
find_121_move(grid)	Scans the board for horizontal and vertical 1-2-1 patterns
is_revealed_number(cell, number)	Validates if a cell is revealed and shows the correct adjacent mine count
check_horizontal_121(grid, start_x, y)	Analyzes the hidden cells directly touching a confirmed 1-2-1 pattern to return a safe “reveal” or a confirmed “flag” action
check_vertical_121(grid, x, start_y)	Analyzes the hidden cells directly touching a confirmed 1-2-1 pattern to return a safe “reveal” or a confirmed “flag” action

File	timer.py
Purpose	Tracks the duration of a game session and stores the player’s best completion time. Starts when the player makes their first move and stops when the game ends.

Functions	
start_timer()	Records the time at the beginning of the game, resets the end time variable
stop_timer()	Stops the timer by recording the current time when the game ends
get_time()	Calculates and returns the number of seconds that have passed since the game started
get_high_score()	Reads and returns saved high score from the local text file
save_high_score(game_time)	Compares current game time against existing high score, saves it if it is a smaller/faster time

Data Flow

The steps below outline the lifecycle of an action within the Minesweeper program. This process repeats continuously in a loop until game_state.py determines if the current session has resulted in a victory or a loss.

Branch 1: Normal Game (Single Player Mode)

1. **Input: interface.py**
    a. The user interacts directly with the Tkinter UI by left-clicking or right-clicking a button widget representing a tile on the 10x10 visual grid.
2. **Routing: interface.py -> main.py**
    a. The widget's bound callback sends the exact grid coordinates (row, col) to handle_left_click(row, col) or handle_right_click(row, col) within the main execution loop.
3. **Update: main.py -> minesweeper.py -> game_state.py**
    a. main.py passes the coordinates to onLeftClick or onRightClick inside minesweeper.py.
    b. minesweeper.py evaluates the game rules (checking if a mine was hit or if a recursive reveal_adjacent_cells() flood-fill is necessary) and directly updates the target Cell object's underlying state variables (is_revealed, is_flagged).
    c. minesweeper.py queries game_state.py to check for win/loss conditions and updates the global status variable.
4. **Display: main.py -> interface.py**
    a. After the backend data structures are updated, main.py pulls the updated states and pushes the grid back to the UI.
    b. interface.py executes the render() function to synchronize the visual Tkinter buttons (raised, sunken, flagged, or numbered) with the new backend states, and updates the status bar counters.

Branch 2: Player vs AI (Interactive Mode)

1. **Input: interface.py**
    c. The user initiates the turn cycle in the Tkinter UI by left-clicking or right-clicking a button widget to make their move on the 10x10 grid.
2. **Routing: interface.py -> main.py**
    d. The user's coordinate is routed to handle_left_click or handle_right_click in main.py to process the human turn.
    e. If the game is still active, main.py schedules the AI's turn using ui.root.after(500, handle_ai_turn) to create a half-second delay.
3. **Update: main.py -> minesweeper.py -> AI Solvers**
    f. main.py calls minesweeper.execute_ai_move().
    g. minesweeper.py queries the active AI solver (easy_solver.py, etc.) via get_next_move(grid).
    h. The AI analyzes the grid and returns the action tuple: (reveal, x, y) or (flag, x, y).
    i. minesweeper.py unpacks the tuple and internally calls its own onLeftClick or onRightClick to update the Cell data states and check game_state.py.

4. Display: main.py -> interface.py

j. After the backend data structures are updated, main.py pulls the updated states and pushes the grid back to the UI.

k. interface.py executes the render() function to synchronize the visual Tkinter buttons (raised, sunken, flagged, or numbered) with the new backend states, and updates the status bar counters.

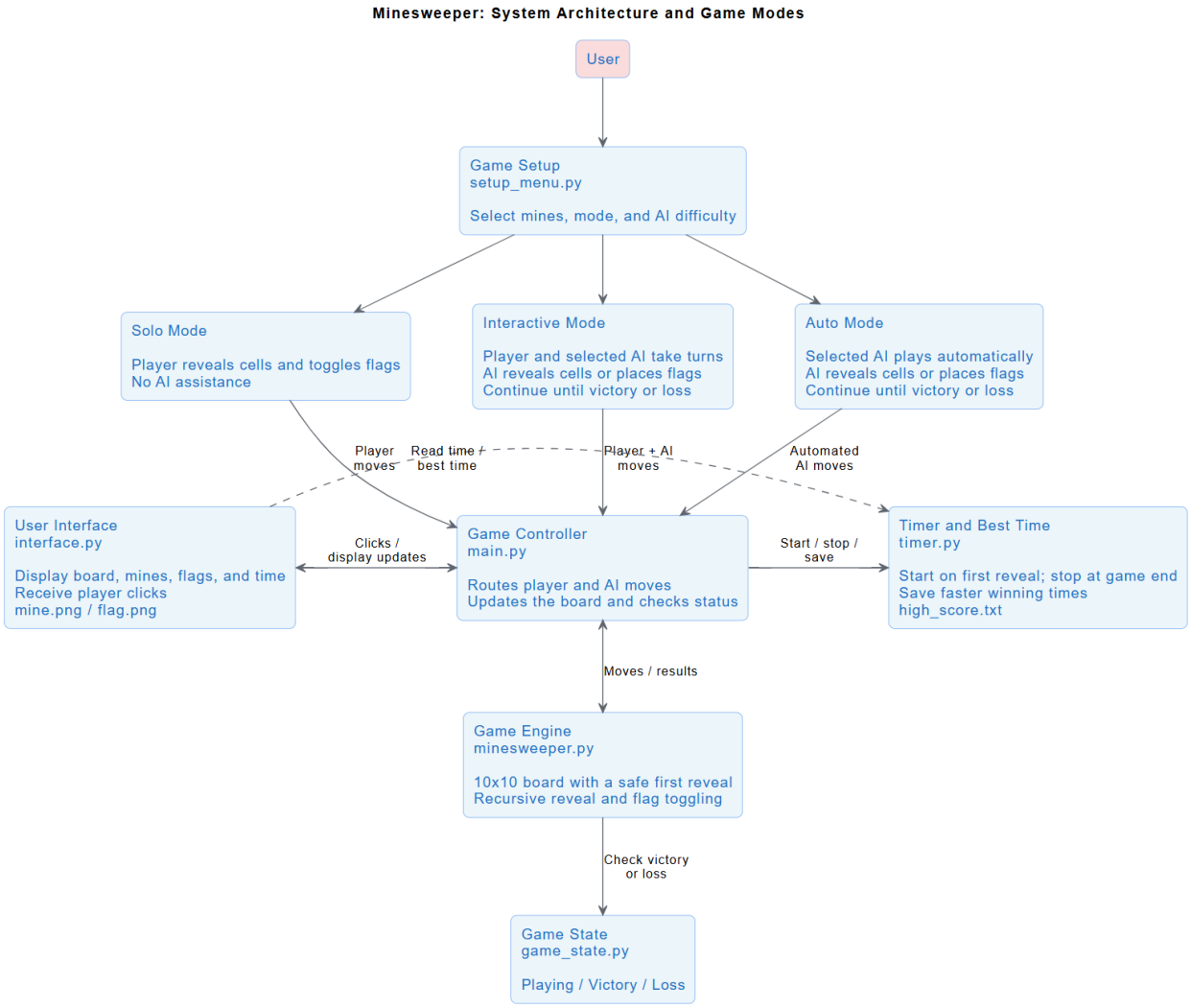
Branch 3: AI Auto Solve Mode

1. **Input: setup_menu.py / main.py**
    a. The user selects “Auto” and an AI difficulty from the pre-game setup window.
    b. Once the board launches, a 1000ms delay triggers handle_ai_turn in main.py.
2. **Routing: main.py -> minesweeper.py -> AI Solvers**
    a. main.py calls minesweeper.execute_ai_move().
    b. minesweeper.py queries the selected AI solver (e.g., easy_solver.py) by passing the current board state to get_next_move(grid).
3. **Update: AI Solvers -> minesweeper.py -> game_state.py**
    a. The AI solver calculates its move and returns an action tuple: (reveal, x, y) or (flag, x, y).
    b. minesweeper.py internally executes its own onLeftClick or onRightClick functions using those coordinates.
    c. This cascades the standard data updates to the Cell objects and triggers game_state.py to check for victory or loss conditions.
4. **Display: main.py -> interface.py**
    a. main.py renders the updated grid via ui.render() in interface.py and updates the status bar.
    b. At the end of handle_ai_turn, if the game is still active (PLAYING), main.py automatically schedules the next AI turn 500ms later using ui.root.after(500, handle_ai_turn). The visuals update rapidly as the bot clears the board step-by-step.

Key Data Structures Table

Data Structure	Python Files	Description
Grid (2D List)	minesweeper.py game_state.py interface.py easy_solver.py medium_solver.py hard_solver.py	A nested 2D array that serves as the core mapping of the board, housing individual Cell objects at specific x/y coords.
Cell (Object)	minesweeper.py	Class instance representing a single square on the board. Holds boolean and integer attributes: - is_mine - is_revealed - is_flagged - adjacent_mines
Configuration (Dictionary)	setup_menu.py main.py	A key-value map generated by the pre-game UI containing: - mines (Integer) - mode (Solo, Interactive, Auto) - difficulty (Easy, Medium Hard)
AI Action Tuple (Tuple)	minesweeper.py easy_solver.py medium_solver.py hard_solver.py	3-element tuple (action, x, y) where action is a string, x and y are integers. Used to pass calculated moves from the solvers back to the game engine.

System Architecture:



Person Hours Estimate Methodology

Our team implemented a Phase-Based Estimation methodology to objectively estimate person-hours. Tasks were categorized by complexity (Easy, Medium, and Hard) and the total estimated time E was calculated using the following formula:

timeU = time to understand requirements
timeC = time to code/work
timeT = time to test/review

E = timeR + timeC + timeT

To ensure accuracy, the coding phase was weighted higher for team members with beginner-level Python experience compared to those with advanced experience.

Task	Assigned	Complexity	Formula	Estimate (hours)
Bot Foundation & Infrastructure	Marie (Beginner)	Hard	timeU = 0.5 timeC = 2 timeT = 0.5	3
Bot Reinforcement Learning Template	Jaydine (Beginner)	Medium	timeU = 0.5 timeC = 1.5 timeT = 0.5	2.5
UI Fixes	Lauren	Medium	timeU = 0.25 timeC = .75 timeT = 0.5	1.5
Bot UI Components	Sabelli	Medium	timeU = 0.5 timeC = 1 timeT = 0.5	2
Easy Solver	Kaitlyn	Easy	timeU = 0.25 timeC = 0.75 timeT = 0.25	1.25
Medium Solver	Greeshma	Medium	timeU = 0.25 timeC = 1 timeT = 0.5	1.75
Hard Solver	Jaydine (Beginner)	Hard	timeU = 0.5 timeC = 1.75 timeT = 0.5	2.75
Custom Enhancements	Ximena	Medium	timeU = 0.5 timeC = 1.25 timeT = 0.5	2.25
System Documentation	Marie	Medium	timeU = 0.5 timeC = 1.5 timeT = 1	3
Github Management	Kaitlyn + Ximena	Easy	timeU = 0.25 timeC = 0.5 timeT = 0.5	1.25

Actual Accounting of Person-Hours

Kaitlyn Bedgood

Day	Task	Hours Worked
9/28	Fork Repo and setup ruleset	1
9/29	Team Meeting w/ TA	1
10/3	Virtual Meeting w/ Team	1
10/4	Updated Git guide document	2
10/5	Implemented easy_solver.py	1
10/6	Team Meeting w/ TA	1
10/7	Research on merge conflicts	1.5
10/9	Updated easy_solver.py	0.5
10/11	Clean up Github	0.5

Marie Biernacki

Day	Task	Hours Worked
9/29	Team meeting w/ TA	1
10/3	Virtual Meeting w/ Team	1
10/4	Code templates and commenting old team's code	3
10/6	Team meeting w/ TA	1
10/8	Bot UI Components + Interactive and Auto Solver integration	1.5
10/9	Bug Fixing	3
10/10	System Documentation	2

Ximena Bustos

Day	Task	Hours Worked
9/29	Team Meeting w/ TA	1
10/3	Virtual Meeting w/ Team	1
10/6	Team meeting w/ TA	1
10/7	Custom addition (adding timer and high score)	6

Sabelli Antebi Delmas

*Had a medical emergency. Did what she was able to for the project.

Day	Task	Hours Worked
9/29	Team Meeting w/ TA	1
10/3	Virtual meeting w/ Team	1

Greeshma Kunduri

Day	Task	Hours Worked
9/29	Team Meeting w/ TA	1
10/3	Virtual Meeting w/ Team	1
10/4		.5
10/5		.5
10/6	Team Meeting w/ TA	1
10/7		.5
10/8		1
10/9		1
10/10		.5

Lauren Lee

Day	Task	Hours Worked
9/29	Team Meeting w/ TA	1
10/3	Virtual Meeting w/ Team	1
10/5	UI Bug Fixed	1
10/6	Team Meeting w/ TA	1
10/7	Testing	1

Jaydine Stiles

Day	Task	Hours Worked
9/29	Team Meeting w/ TA	1
10/3	Virtual Meeting w/ Team	1
10/5	Bot Solver Templates	1
10/6	Team Meeting w/ TA	1
10/8	Hard Solver	2
10/9	Bug Fixing	3
10/11	System documentation/final code cleanup	2